

ASE WEEK 02 — UNIT 01

SDLC Fundamentals

The software life cycle, the process, its four activities, and why a team cannot work without one

Sources. Sommerville, *Software Engineering* 9th ed., pp.44–46. Mall, *Fundamentals of Software Engineering* 4th ed., pp.67–71. **Page numbers are PDF page numbers**, not printed page numbers — the front-matter offset differs between the two books. **Note on method.** This is a compendium, not a summary. Every definition is quoted verbatim from the source before it is explained, and every claim carries a page anchor. Where a term carries more than one meaning in the literature, both are given.

Where this sits

الوحدة الأولى: المفاهيم التأسيسية لدورة حياة وبرمجيات الفريق.

The entire series rests on one fact: **professional software is built by teams, not by individuals.** A single programmer writing a small program can succeed with no process at all. A team cannot — and the reason is not skill, it is coordination. This file establishes that, and it establishes the vocabulary the remaining nine files will use.

The question this file answers: *why does a software team need a defined process in the first place?*

السؤال الذي تسلمه هذه الوحدة إلى Unit 02: سلّمنا بأن العملية ضرورية؛ فكيف ينبغي أن تبدو أول عملية منظّمة؟ الجواب البديهي هو «خطّط لكل شيء مقدماً»، وهذا الجواب له اسم شهير: **The Waterfall Model**.

1. The software life cycle

1.1 The analogy the term is built on

Mall does not open with a definition. He opens with a comparison, because the term itself was built on one.

Verbatim (Mall p.67): "It is well known that all living organisms undergo a life cycle. For example when a seed is planted, it germinates, grows into a full tree, and finally dies. Based on this concept of a biological life cycle, the term software life cycle has been defined to imply the different stages (or phases) over which a software evolves from an initial customer request for it, to a fully developed software, and finally to a stage where it is no longer useful to any user, and then it is discarded."

الشرح المفاهيمي: Mall ما يبلش بتعريف — يبلش بتشبيه، لأن المصطلح نفسه مبني على تشبيه. كل كائن حي إله دورة حياة: بذرة تنبت، تكبر وتصير شجرة، وبالأخر تموت. وعلى هذا الأساس عُرّف الـ **software life cycle**: المراحل اللي يمر بيها البرنامج من طلب العميل الأول، إلى برنامج مكتمل، إلى مرحلة يصير ما يفيد أحد فيُرمى.

ليش التشبيه مهم مو مجرد مقدمة؟ لأنه يفرض ثلاث نتائج: 1. البداية طلب، مو برنامج — البرنامج ما موجود بالسؤال الأصلي. 2. فيه مراحل متتالية محدّدة — مو عملية واحدة مستمرة. 3. فيه نهاية — البرنامج يُرمى. يعني البرنامج ما يعيش للأبد، ولهذا

الصيانة والتطوير جزء من دورته مو استثناء.

1.2 The stages, in the source's own order

Mall names the stages as the software moves through them:

Stage	Mall's wording
Inception	"This stage where the customer feels a need for the software and forms rough ideas about the required features is known as the inception stage."
Development stages	"Starting with the inception stage, a software evolves through a series of identifiable stages (also called phases) on account of the development activities carried out by the developers, until it is fully developed and is released to the customers."
Operation (also called maintenance)	"Once installed and made available for use, the users start to use the software. This signals the start of the operation (also called maintenance) phase."
Retirement	"Finally the software is retired, when the users do not find it any longer useful..."

المعنى بالعربية لمراحل دورة الحياة:

المعنى	المرحلة
العميل يحسّ بحاجة ويكوّن أفكاراً خشنة عن الميزات المطلوبة	Inception
سلسلة مراحل متتالية بحسب أنشطة المطورين، لحد ما يصير مكتملاً ويُسلّم	مراحل التطوير
بعد التنصيب والاستخدام — Mall يعتبرها نفس المرحلة باسمين	Operation (= Maintenance)
يُرمى لما يصير ما يفيد أحد	Retirement

نقطة انتبه لها: Mall يسميها **operation (also called maintenance)** — يعني التشغيل والصيانة مرحلة واحدة بوجهين. لو سألك «شنو مرحلة الصيانة؟» — هي نفسها مرحلة التشغيل.

1.3 The inception stage — the fact that makes the whole series necessary

This is the single most consequential sentence in the file:

Verbatim (Mall p.67): "At this stage, the customers are usually not clear about all the features that would be needed, neither can they completely describe the identified features in concrete terms, and can only vaguely describe what is needed."

التحليل الهندسي لأهمية البداية المبهمة: العميل بمرحلة البداية: - مو واضح شنو يريد كل الميزات، - ما يكدر يوصف الميزات اللي يعرفها بوصف ملموس، - يكدر يوصف المطلوب بشكل مبهم فقط.

لبش هذه أساس السلسلة؟ لأنها تعني إن المطلوبات ناقصة ومبهمة من اليوم الأول — مو لأن العميل مقصّر، بل لأن هذا طبيعة المرحلة. ومن هنا تنبع كل مشكلة بكل نموذج:

- Waterfall يفترض المطلوبات معروفة ومستقرة → يخالف هذه الحقيقة.
- Prototyping يحلّها بالبناء لاستكشاف المطلوبات.
- Incremental يحلّها بتسليم شرائح تتعلم منها.

- Agile يحلّها بدورات قصيرة.

يعني: كل نموذج بالملفات الجاية هو محاولة للتعامل مع هذه الجملة.

1.4 The maintenance phase — why it is the longest

Verbatim (Mall p.67–68): "As the users use the software, not only do they request for fixing any failures that they might encounter, but they also continually suggest several improvements and modifications to the software. Thus, the maintenance phase usually involves continually making changes to the software to accommodate the bug-fix and change requests from the user."

Verbatim (Mall p.68): "The operation phase is usually the longest of all phases and constitutes the useful life of a software."

الصيانة مو بس إصلاح أعطال. المستخدمون: - يطلبون إصلاح إخفاقات، و - يقترحون تحسينات وتعديلات باستمرار.

فالصيانة = تغييرات مستمرة لتلبية طلبات الإصلاح وطلبات التغيير معاً.

والنقطة الكمية: "The operation phase is usually the longest of all" — مرحلة التشغيل أطول مرحلة. يعني الجزء الأكبر من عمر البرنامج يستهلكه التغيير، مو البناء الأول.

وهذا يبرّر السلسلة كلها: لو كانت أطول مرحلة هي التغيير، فالنموذج اللي ما يتعامل وياها زين نموذج فاشل — بغض النظر عن أناقته في البناء الأول.

1.5 Retirement — and why it happens

Verbatim (Mall p.68): "Finally the software is retired, when the users do not find it any longer useful due to reasons such as changed business scenario, availability of a new software having improved features and working, changed computing platforms, etc."

ثلاثة أسباب للتقاعد: 1. تغيّر سيناريو العمل — النشاط نفسه تغيّر. 2. توقّر برنامج جديد بميزات وتشغيل أفضل. 3. تغيّر منصات الحوسبة.

ليش يذكرها؟ لأنها تُظهر إن التقاعد مو فشل — مرحلة طبيعية. والسبب الثاني مهم: برنامج أفضل يقتل برنامجاً شغّالاً. يعني المنافسة على الجودة مو ترف.

1.6 The formal definition

Verbatim (Mall p.68): "The life cycle of a software represents the series of identifiable stages through which it evolves during its life time."

التعريف الرسمي: سلسلة مراحل قابلة للتمييز يمر بيها البرنامج خلال عمره.

احفظ كلمة **identifiable** — المراحل مُعرّفة ومميّزة، مو استمرارية مبهمة. هذا اللي يجعل «النموذج» ممكنأً أصلاً: لو ما كدرنا نميّر المراحل، ما كدرنا نرسمها ولا نرتّبها.

2. The software process and its four activities

2.1 The definition

Verbatim (Sommerville p.45): "A software process is a set of related activities that leads to the production of a software product."

And the qualification that matters for the rest of the series:

Verbatim (Sommerville p.45): "These activities may involve the development of software from scratch in a standard programming language like Java or C. However, business applications are not necessarily developed in this way. New business software is now often developed by extending and modifying existing systems or by configuring and integrating off-the-shelf software or system components."

الـ **software process** = مجموعة أنشطة مترابطة تؤدي لمنتج برمجي.

بس لاحظ التحذير اللي يضيفه فوراً: البرامج مو دائماً تُبنى من الصفر. البرامج التجارية الحديثة كثيراً ما تُبنى بـ - توسيع وتعديل أنظمة موجودة، أو - تهيئة ودمج برامج أو مكونات جاهزة.

ليش يهمننا؟ لأنه بيترّ الملف 03 (التطوري) والملف 05 (RAD)، وأساسه إعادة استخدام الكود). يعني الفكرة اللي تنبني عليها RAD مذكورة من أول فصل.

2.2 The four fundamental activities — the invariant

Verbatim (Sommerville p.45): "There are many different software processes but all must include four activities that are fundamental to software engineering:"

#	Activity	Verbatim definition
1	Software specification	"The functionality of the software and constraints on its operation must be defined."
2	Software design and implementation	"The software to meet the specification must be produced."
3	Software validation	"The software must be validated to ensure that it does what the customer wants."
4	Software evolution	"The software must evolve to meet changing customer needs."

Verbatim (Sommerville p.45): "In some form, these activities are part of all software processes."

And they are not atomic:

Verbatim (Sommerville p.45): "In practice, of course, they are complex activities in themselves and include sub-activities such as requirements validation, architectural design, unit testing, etc. There are also supporting process activities such as documentation and software configuration management."

التحليل المفاهيمي للأنشطة الأربعة: أربعة أنشطة موجودة بكل عملية مهما كان شكلها:

#	النشاط	المعنى
1	Specification	تحديد الوظائف والقيود
2	Design & implementation	إنتاج البرنامج الذي يحقق المواصفة
3	Validation	التأكد إنه يسوي الذي يريده العميل
4	Evolution	يتطور مع تغير احتياجات العميل

النشاط الرابع هو بيت القصيد. لاحظ إن **evolution** نشاط أساسي، مو ملحق. يعني التغيير مدمج بالتعريف نفسه — وهذا ثالث مرة نشوف نفس الفكرة (بعد inception المبهم وطول مرحلة التشغيل). تتكرر بثلاث صيغ مختلفة لأنها جوهر الموضوع. وكل واحد منها مو ذرة: بيه أنشطة فرعية (التحقق من المطلوبات، التصميم المعماري، اختبار الوحدة)، وفيه أنشطة سائدة: التوثيق وإدارة تهيئة البرمجيات.

احفظ الأربعة بترتيبها — كل نموذج بالملفات الجاية هو ترتيب مختلف لهذه الأربعة. هذا مفتاح المقارنة.

3. What a process description actually contains

Verbatim (Sommerville p.45): "When we describe and discuss processes, we usually talk about the activities in these processes such as specifying a data model, designing a user interface, etc., and the ordering of these activities. However, as well as activities, process descriptions may also include:"

Element	Verbatim
Products	"which are the outcomes of a process activity. For example, the outcome of the activity of architectural design may be a model of the software architecture."
Roles	"which reflect the responsibilities of the people involved in the process. Examples of roles are project manager, configuration manager, programmer, etc."
Pre- and post-conditions	"which are statements that are true before and after a process activity has been enacted or a product produ[ced]."

العناصر الثلاثة الإضافية في توصيف العملية: العملية مو بس أنشطة وترتيبها، توصيفها يحتوي ثلاثة عناصر إضافية:

العنصر	المعنى	مثال
Products	مخرجات النشاط	التصميم المعماري → موديل المعمارية
Roles	مسؤوليات الناس	مدير مشروع · مدير تهيئة · مبرمج

العنصر	المعنى	مثال
Pre/post-conditions	شروط صحيحة قبل وبعد النشاط	—

يجي كسؤال مباشر: «شنو يوصف العملية؟» — الأنشطة + ترتيبها + المنتجات + الأدوار + الشروط القبلية والبعدية. لو ذكرت الأنشطة بس، الجواب ناقص.

4. The three-level vocabulary — SDLC, process, methodology

This is the part most students blur. Mall draws **two distinct distinctions** that together form a three-level hierarchy.

4.1 Level one: SDLC versus process

Verbatim (Mall p.68): "A software development life cycle (SDLC) model (also called software life cycle model and software development process model) describes the different activities that need to be carried out for the software to evolve in its life cycle."

Verbatim (Mall p.68): "Throughout our discussion, we shall use the terms software development life cycle (SDLC) and software development process interchangeably. However, some authors distinguish an SDLC from a software development process. In their usage, a software development process describes the life cycle activities more precisely and elaborately, as compared to an SDLC. Also, a development process may not only describe various activities that are carried out over the life cycle, but also prescribe specific methodologies to carry out the activities, and also recommends the specific documents and other artifacts that should be produced at the end of each phase. In this sense, the term SDLC can be considered to be a more generic term, as compared to the development process."

Verbatim (Mall p.69): "...several development processes may fit the same SDLC."

الشرح والتمييز الأول بين SDLC والعملية: Mall يعطيك الاسم بثلاث تسميات مترادفة: **software life cycle = SDLC model** و **software development process model = model**.

بس بعدين يسجل فرقاً مهماً: هو شخصياً يستخدم SDLC و«عملية التطوير» بالتبادل، بس بعض المؤلفين يفرّقون: - عملية التطوير تصف الأنشطة بدقة وتفصيل أكثر، - وقد توصي بمنهجيات محددة، - وقد توصي بوثائق ومخرجات محددة تُنتج بنهاية كل مرحلة. - فيصير SDLC مصطلحاً أعم.

والنتيجة العملية: «عدة عمليات تطوير ممكن تنسجم مع نفس الـ SDLC» — الـ SDLC هيكل واحد، وفوقه ممكن عدة عمليات.

4.2 Level two: process versus methodology

Verbatim (Mall p.69): "Though the terms process and methodology are at time used interchangeably, there is a subtle difference between the two. First, the term process has a broader scope and addresses either all the activities taking place during software development, or certain coarse grained activities such as design (e.g. design process), testing (test process), etc. Further, a software process not only identifies the specific activities that need to be carried out, but may also prescribe certain methodology for carrying out each activity."

Verbatim (Mall p.69): "A methodology, on the other hand, prescribes a set of steps for carrying out a specific life cycle activity. It may also include the rationale and philosophical assumptions behind the set of steps through which the activity is accomplished."

Verbatim (Mall p.69): "A process usually describes all the activities starting from the inception of a software to its maintenance and retirement stages, or at least a chunk of activities in the life cycle. It also recommends specific methodologies for carrying out each activity. A methodology, in contrast, describes the steps to carry out only a single or at best a few individual activities."

Mall's own example:

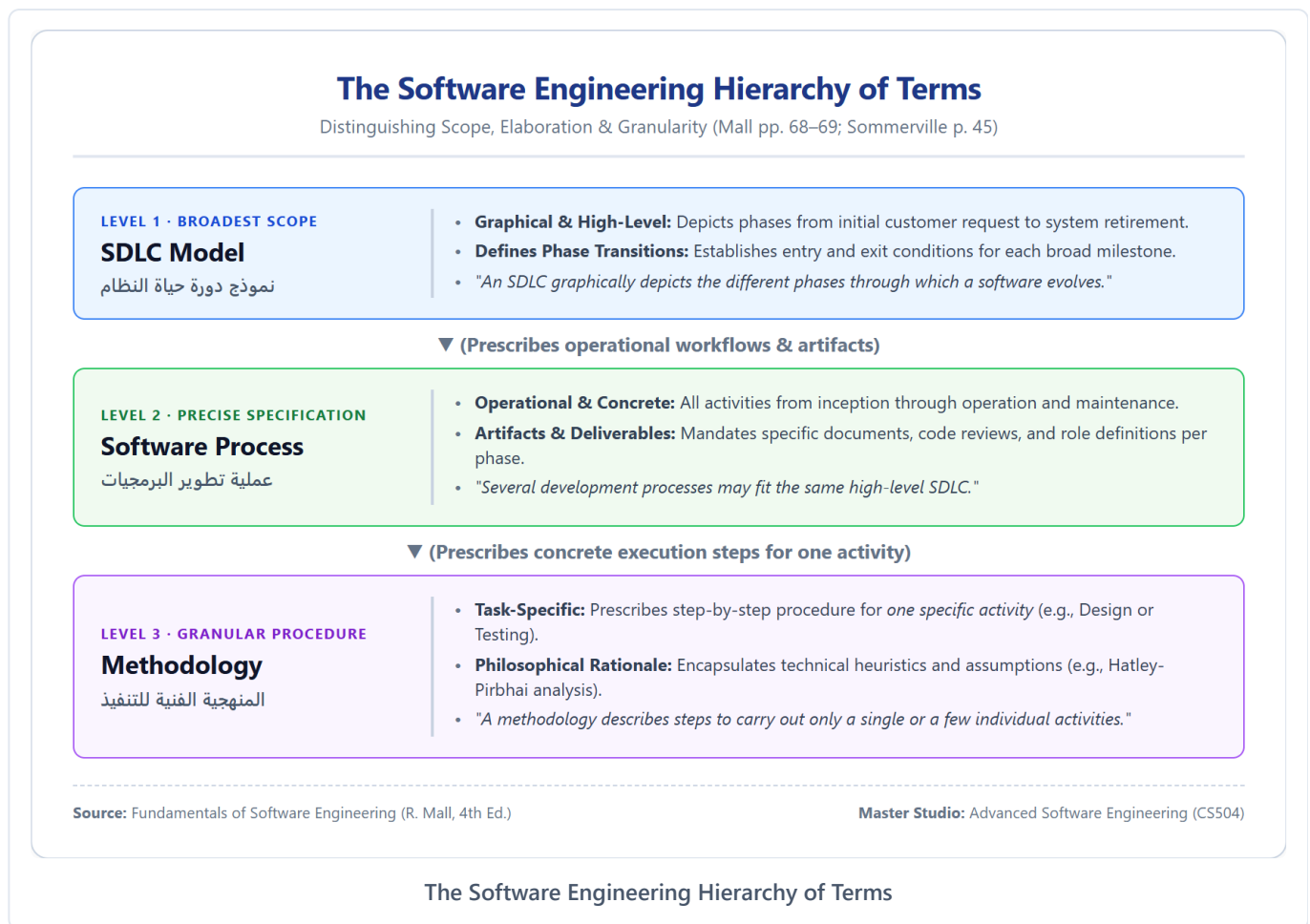
Verbatim (Mall p.69): "For example, a design process may recommend that in the design stage, the high-level design activity be carried out using Hatley and Pirbhai's structured analysis and design methodology."

المقارنة التفصيلية بين العملية والمنهجية:

المنهجية) Methodology	Software Process (العملية)	المعيار
نشاط واحد أو بضعة أنشطة	أوسع — كل الأنشطة أو أنشطة كبيرة (عملية التصميم، عملية الاختبار)	النطاق
خطوات نشاط واحد	من البداية (inception) للتشغيل والتقاعد	يغطي
—	بمنهجية لكل نشاط	قد يوصي
المبّرر والافتراضات الفلسفية وراء الخطوات	—	قد يشمل

مثال Mall: «عملية التصميم» ممكن توصي إن التصميم العالي يُسوَّى بمنهجية (Hatley and Pirbhai's structured analysis and design).

4.3 The hierarchy, put together



التدرج المفاهيمي (من الأعمّ إلى الأخصّ):

SDLC Model → Software Process → Software Methodology

للامتحان: لو سألك «الفرق بين SDLC والعملية؟» أو «بين العملية والمنهجية؟» — المنطق واحد: الأعمّ مقابل الأخصّ.

4.4 The SDLC is drawn, not only described

Verbatim (Mall p.69): "An SDLC is represented graphically by drawing various stages of the life cycle and showing the transitions among the phases. This graphical model is usually accompanied by a textual description of various activities that need to be carried out during a phase before that phase can be considered to be complete."

Verbatim (Mall p.69): "An SDLC graphically depicts the different phases through which a software evolves. It is usually accompanied by a textual description of the different activities that need to be carried out during each phase."

الرسم المقترن بالنص التوصيفي: الـ SDLC يُرسم — يبيّن المراحل والانتقالات بينها، ويُرافق الرسم وصف نصي للأنشطة التي لازم تُنجز قبل اعتبار المرحلة مكتملة.

عنصران معاً: رسم + نص. الرسم جزء من التعريف مو زينة — ولهذا كل نموذج بالملفات الجاية إله شكل، ولهذا ندرس الأشكال.

5. Why a process is needed — the argument

5.1 The stated advantage

Verbatim (Mall p.69): "The primary advantage of using a development process is that it encourages development of software in a systematic and disciplined manner. Adhering to a process is especially important to the development of professional software needing team effort."

Verbatim (Mall p.70): "When software is developed by a team rather than by an individual programmer, use of a life cycle model becomes indispensable for successful completion of the project."

Verbatim (Mall p.70): "Software development organisations have realised that adherence to a suitable life cycle model helps to produce good quality software and that helps minimise the chances of time and cost overruns."

التحليل الهندسي لأهمية العملية: الفائدة الأساسية: تطوير منهجي ومنضبط. والنتيجتان الملموستان: 1. جودة أفضل، 2. تقليل احتمال تجاوز الوقت والكلفة.

واحفظ الكلمة القوية: "indispensable" — لا غنى عنه للشغل الفريقي. مو «مفيد» — لا غنى عنه.

5.2 programming-in-the-small versus programming-in-the-large

Verbatim (Mall p.70–71): "Programming-in-the-small refers to development of a toy program by a single programmer. Whereas programming-in-the-large refers to development of a professional software through team effort."

Verbatim (Mall p.71): "While development of a software of the former type could succeed even while an individual programmer uses a build and fix style of development, use of a suitable SDLC is essential for a professional software development project involving team effort to succeed."

المقارنة الحاسمة بين البرمجة الفردية والبرمجة الفريقية:

Programming-in-the-large	Programming-in-the-small	وجه المقارنة
فريق هندسي	مبرمج واحد	القائم بالعمل
برنامج احترافي معقد	برنامج لعبة (toy program)	طبيعة المنتج
نعم — استخدام SDLC حتمي وإلزامي	لا — ممكن ينجح بـ build and fix	هل يحتاج SDLC؟
		مثال Mall للصغير: طالب يحل واجب صف — ممكن ينجح بلا عملية.

ليش مهم للامتحان؟ لأنه يمنع الجواب الخاطئ: لو سألك «هل نحتاج SDLC دائماً؟» — الجواب مو نعم مطلقاً. الجواب: يعتمد على الحجم. للصغير لا، وللأكبر ضروري.

5.3 What actually goes wrong without a process

Verbatim (Mall p.70): "Suppose, a software development problem has been divided into several parts and these parts are assigned to the team members. From then on, suppose the team members are allowed the freedom to develop the parts assigned to them in whatever way they like. It is possible that one member might start writing the code for his part while making assumptions about the input results required from the other parts, another might decide to prepare the test documents first, and some other developer might start to carry out the design for the part assigned to him. In this case, severe problems can arise in interfacing the different parts and in managing the overall development."

Verbatim (Mall p.70): "Therefore, ad hoc development turns out to be is a sure way to have a failed project. Believe it or not, this is exactly what has caused many project failures in the past!"

سيناريو الفوضى التطويرية بدون عملية: السيناريو الملموس: المشكلة تنقسم أجزاء وتوزع على الأعضاء، ويترك لكل واحد حرية الطريقة. النتيجة المحتملة:

- واحد يبيلش يكتب الكود وهو يفترض شنو راح تكون مخرجات الأجزاء الثانية،
- وواحد يبيلش بمستندات الاختبار أول،
- وواحد يبيلش بالتصميم.

والنتيجة: مشاكل شديدة في الواجهات بين الأجزاء وفي إدارة التطوير ككل.

والحكم: التطوير العشوائي = طريق مضمون لمشروع فاشل، وهذا بالضبط سبب كثير من فشل المشاريع بالماضي.

لاحظ وين يتركز الفشل: مو بالكود نفسه — بل بالواجهات والإدارة. يعني مشكلة تنسيق مو مشكلة مهارة. نقطة تحليلية تنفع للسيناريوهات.

5.4 The one-sentence version of the argument

Verbatim (Mall p.70): "When a software is developed by a team, it is necessary to have a precise understanding among the team members as to—when to do what. In the absence of such an understanding, if each member at any time would do whatever activity he feels like doing. This would be an open invitation to developmental chaos and project failure."

خلاصة الحجة في عبارة واحدة:

«تفاهم دقيق بين أعضاء الفريق: متى نسوي شنو.»

بدون هذا التفاهم → كل واحد يسوي اللي يحسّه → دعوة مفتوحة للفوضى التطويرية وفشل المشروع.
وهذا التعريف العملي للـSDLC: هو اللي يحدد متى نسوي شنو. لو نسيت كل شي بالملف، احفظ هذه.

6. Why the process must be *documented*

Verbatim (Mall p.71): "It is not enough for an organisation to just have a well-defined development process, but the development process needs to be properly documented."

Verbatim (Mall p.71): "In this case, its developers develop only an informal understanding of the development process. An informal understanding of the development process among the team members can create several problems during development."

Verbatim (Mall p.71): "A documented process model ensures that every activity in the life cycle is accurately defined. Also, wherever necessary the methodologies for carrying out the respective activities are described. Without documentation, the activities and their ordering tend to be loosely defined, leading to confusion and misinterpretation by different teams in the organisation."

Verbatim (Mall p.71): "For example, code reviews may informally and inadequately be carried out since there is no documented methodology as to how the code review should be done. Another difficulty is that for loosely defined activities, the developers tend to use their subjective judgments. As an example, unless it is explicitly prescribed, the team members would subjectively decide as to whether the test cases should be designed just after the requirements phase, after the design phase, or after the coding phase. Also, they would debate whether the test cases should be documented at all and the rigour with it should be documented."

Verbatim (Mall p.71): "An undocumented process gives a clear indication to the members of the development teams about the lack of seriousness on the part of the management of the organisation about following the process."

مو كافي يكون عندك عملية — لازم تكون موثقة. بدون توثيق، المطورين يكوّنون فهم غير رسمي فقط، وهذا يخلق مشاكل:

المثال اللي يعطيه Mall	المشكلة
→ التباس وسوء تفسير بين فرق مختلفة	الأنشطة وترتيبها تصير فضفاضة
→ مراجعات الكود تُسوّى بشكل غير رسمي وناقص	ما يوجد منهجية موثقة
→ متى تُصمّم حالات الاختبار: بعد المطلوبات؟ بعد التصميم؟ بعد الكود؟	المطورون يستخدمون أحكامهم الذاتية
→ هل تُوثّق حالات الاختبار أصلاً؟ وبأي صرامة؟	نقاشات بلا مرجع

والنتيجة الثقافية — وهذي مهمة: العملية غير الموثقة تُرسل إشارة واضحة للفرق بعدم جدية الإدارة تجاه اتباع العملية.

يعني التوثيق مو إجراء إداري — هو جزء من العملية نفسها. عملية غير موثقة = عملية غير موجودة عملياً.

7. The two poles — plan-driven and agile

Verbatim (Sommerville p.46): "Sometimes, software processes are categorized as either plan-driven or agile processes. Plan-driven processes are processes where all of the process activities are planned in advance and progress is measured against this plan. In agile processes, which I discuss in Chapter 3, planning is incremental and it is easier to change the process to reflect changing customer requirements."

Verbatim (Sommerville p.46): "As Boehm and Turner (2003) discuss, each approach is suitable for different types of software. Generally, you need to find a balance between plan-driven and agile processes."

Verbatim (Sommerville p.46): "For critical systems, a very structured development process is required. For business systems, with rapidly changing requirements, a less formal, flexible process is likely to be more effective."

Agile	Plan-driven	
تخطيط تزايدى	كل الأنشطة مخططة مسبقاً	التخطيط
—	على الخطة	قياس التقدم
أسهل — العملية تتغير لتعكس متطلبات متغيرة	صعب	التغيير

والقاعدة الحاسمة: «كل أسلوب يناسب أنواعاً مختلفة من البرامج» (Boehm and Turner, 2003) — ما موجود فائز. تحتاج توازن.

الأسلوب	نوع النظام
عملية منظمة جداً	Critical systems
عملية أقل رسمية وأكثر مرونة	Business systems بمتطلبات سريعة التغير

بذرة الملف 09 — احفظ الجدول، لأنه يجي بصيغة «أي نموذج يناسب هذا المشروع؟».

8. The thread — what the remaining nine files are about

Sommerville states this chapter's own objectives (p.44):

Verbatim: "When you have read this chapter you will: - understand the concepts of software processes and software process models; - have been introduced to three generic software process models and when they might be used; - know about the fundamental process activities of software requirements engineering, software development, testing, and evolution; - understand why processes should be organized to cope with changes in the software requirements and design; - understand how the Rational Unified Process integrates good software engineering practice to create adaptable software processes."

And Mall states the same concern from the historical side (p.67):

Verbatim: "The genesis of the agile model can be traced to the radical changes to the types of project that are being undertaken at present, rather than to any radical innovations to the life cycle models themselves. The projects have changed from large multi-year product development projects to small services projects now."

Mall also lays out his chapter as a narrative (p.67):

Verbatim: "...we discuss a few derivatives of this model. Subsequently we discuss the spiral model that generalises various life cycle models. Finally, we discuss a few recently proposed life cycle models that are categorized under the umbrella term agile model."

الرابط الفلسفي بين النماذج العشرة: Sommerville يحدد هدف الفصل بخمسة أهداف، وأهم اثنين: ثلاثة نماذج عملية عامة ومتى تُستخدم، وليس لازم تُنظّم العمليات حتى تتحمّل التغيّرات.

وMall يقول شي أعمق: ظهور Agile ما جاء من اختراع جديد — جاء من تغيّر نوع المشاريع نفسها: من مشاريع منتج كبيرة تدوم سنوات إلى مشاريع خدمات صغيرة.

وMall نفسه يعرض الفصل كتسلسل: waterfall → مشتقاته → spiral اللي يعمّم النماذج → agile. الخيط اللي يربط الملفات العشرة:

المطلوبات تبدأ مبهمّة (inception)، والتغيير هو أطول مرحلة (maintenance)، وevolution نشاط أساسي. يعني التغيير مو استثناء — هو القاعدة. وكل نموذج بالملفات الجاية هو جواب على سؤال واحد: متى وكيف نتحمّل التغيير؟

النماذج كأجوبة — هكذا اقرأ التسعة الجاية:

الوحدة / النموذج	الجواب الهندسي الذي يقّده لاستيعاب التغيير
Unit 02 (Waterfall)	قزّر كل شي مقدماً — ينجح حصراً لما تكون المطلوبات مستقرة تماماً
Unit 03 (Prototyping & Evolutionary)	اكتشف المطلوبات بالبناء التفاعلي (Prototyping) ودع النظام يتطور تدريجياً (Evolutionary)
Unit 04 (Incremental)	سلّم شرائح تشغيلية واستوعب التغيير بين كل دورة تسليم
Unit 05 (RAD)	اضغط الجدول الزمني بالتطوير السريع المبني على المكونات الجاهزة
Unit 06 (Spiral)	خلّي تحليل المخاطر (Risk) هو الموجه الأساسي لكل حلقة تطوير
Unit 07 (Unified Process)	اجمع الكل بإطار مرحلي وتكراري موحد (Construction, Elaboration, Inception, Transition)
Unit 08 (Agile / XP / Scrum)	غيّر الفلسفة بالكامل: الأفراد والتفاعل قبل الإجراءات والوثائق
Unit 09 (Model Selection)	لا يوجد نموذج مثالي مطلق — تعلّم معايير الاختيار الهندسي بحسب المخاطر والمطلوبات
Unit 10 (Master Synthesis)	مصفوفة المقارنة الشاملة وبنك أسئلة الامتحان التحليلية

Source notes

Source	Verdict for this file
Mall p.67–71	Primary for §1, §4, §5 and §6. The life-cycle definition and stages, the inception stage, operation/maintenance and retirement, the SDLC definition and synonyms, both vocabulary distinctions (SDLC/process and process/methodology), programming-in-the-small/large, the team-failure scenario, and the documentation argument are all Mall's. Sommerville has none of them.
Sommerville p.44–46	Primary for §2, §3 and §7. The process definition, the four fundamental activities, the products/roles/pre-conditions list, the off-the-shelf qualification, and the plan-driven/agile axis are all his, all verbatim. Mall has none of them.
Pressman / Agarwal	Add nothing to this file. Both restate the four activities without Mall's vocabulary precision or Sommerville's process-description detail.
Textbook Scope Note	The three-level vocabulary hierarchy (§4) rests on Mall alone. Sommerville draws neither distinction. If the doctor asks about SDLC vs process or process vs methodology, Mall p.68–69 is the only authority you have — and note Mall explicitly says he uses the terms interchangeably himself, so the distinction is a <i>reported usage</i> , not his own.
Textbook Scope Note	programming-in-the-small / programming-in-the-large appears at Mall p.70–71, defining the exact organizational boundary where team coordination makes an SDLC strictly indispensable.
Note	Sommerville's objective mentions "three generic software process models" — waterfall, incremental development, and reuse-oriented software engineering (Sommerville p.47). This is a different taxonomy from the lecture's , which follows Mall. Both are correct; they are different cuts. Flagged so the difference is not mistaken for a contradiction.

Retrieval set

Answers appear directly beneath each question, as agreed. **Cover the answer, produce your own, then compare.**

1 Why does the software life cycle begin with a request rather than with software?

Because the term is defined on the biological analogy: it runs from "*an initial customer request*" to the point where the software is "*no longer useful to any user, and then it is discarded.*" The request is the seed. (Mall p.67)

2 Name the four stages Mall identifies, and say which is the longest.

Inception → the development stages → **operation (also called maintenance)** → **retirement**. "*The operation phase is usually the longest of all phases and constitutes the useful life of a software.*" (Mall p.67–68)

3 What exactly is wrong with the requirements at the inception stage?

The customers are *"not clear about all the features that would be needed"*, cannot *"completely describe the identified features in concrete terms"*, and *"can only vaguely describe what is needed."* (Mall p.67)

4 Name the four fundamental activities of a software process, in order, and state what makes them fundamental.

Specification · design and implementation · validation · evolution. They are fundamental because *"In some form, these activities are part of all software processes."* (Sommerville p.45)

5 A process description is not only activities. What else does it contain? Give one example of each.

Products (the outcome of architectural design is a model of the software architecture) · **roles** (project manager, configuration manager, programmer) · **pre- and post-conditions** (statements true before and after an activity is enacted). (Sommerville p.45)

6 Distinguish SDLC, process and methodology — widest to narrowest.

SDLC is the most generic: the phases a software evolves through, graphically depicted plus textually described. A **process** is more precise and elaborate: it describes all activities from inception to maintenance and retirement, may prescribe methodologies, and may name the documents produced per phase — and *"several development processes may fit the same SDLC."* A **methodology** prescribes the steps for *"only a single or at best a few individual activities"*, and may include the rationale and philosophical assumptions behind those steps. (Mall p.68–69)

7 Distinguish programming-in-the-small from programming-in-the-large, and say what follows for the use of an SDLC.

Small = a *"toy program by a single programmer"*; **large** = *"professional software through team effort."* For small work a build-and-fix style can succeed; for large work *"use of a suitable SDLC is essential."* So the answer to "is an SDLC always needed?" is **no — it depends on scale.** (Mall p.70–71)

8 Describe what goes wrong when a team has no process. Where does the failure actually appear?

Members work to their own assumptions: one writes code *"while making assumptions about the input results required from the other parts"*, another prepares test documents first, another starts designing. *"Severe problems can arise in interfacing the different parts and in managing the overall development."* The failure lands on **coordination**, not on coding skill. (Mall p.70)

9 State, in one sentence, the understanding a team needs — and what its absence produces.

A "precise understanding among the team members as to — when to do what." Without it, each member does whatever he feels like, which is "an open invitation to developmental chaos and project failure." (Mall p.70)

10 Having a process is not enough. What else is required, and why?

It must be **properly documented**. Without documentation the team holds only "an informal understanding", activities and their ordering become "loosely defined", developers fall back on **subjective judgment** (e.g. when to design test cases, or whether to document them at all), and — importantly — an undocumented process signals to the team "the lack of seriousness on the part of the management." (Mall p.71)

11 Define plan-driven and agile processes, and say which wins.

Plan-driven: all process activities are planned in advance and progress is measured against the plan. **Agile:** planning is incremental and the process is easier to change. **Neither wins** — "each approach is suitable for different types of software" (Boehm and Turner, 2003), and a balance is needed. Critical systems need structure; business systems with rapidly changing requirements need flexibility. (Sommerville p.46)

12 State the one problem the whole series of models is trying to solve, and why the series is a story rather than a list.

That **requirements begin vague and then change** — inception is unclear, maintenance (the longest phase) is continual change, and evolution is one of the four fundamental activities. Every model that follows is a different answer to *when* and *how* the change is absorbed. (Sommerville p.44–45; Mall p.67–68)

Page anchors

Revisit	For
Sommerville p.44	The five chapter objectives — including "three generic models" and the "cope with change" thread
Sommerville p.45	Software process definition · the four activities · sub-activities and supporting activities · products/roles/pre-post-conditions · the off-the-shelf qualification
Sommerville p.46	Software process model definition · process improvement and standardisation · plan-driven vs agile · the Boehm and Turner balance point · critical vs business systems
Mall p.67	The chapter's own narrative plan · the agile genesis (project types changed) · the biological analogy · the life-cycle definition · the inception stage and the vague customer · the operation/maintenance phase

Revisit	For
Mall p.68	Maintenance as continual bug-fix and change requests · operation is the longest phase · retirement and its three causes · the one-line life-cycle definition · the SDLC definition and its three synonyms · SDLC vs process (reported distinction)
Mall p.69	"several development processes may fit the same SDLC" · the graphical SDLC definition · process vs methodology in full · the Hatley and Pirbhai example · why use a development process
Mall p.70	Life cycle model is <i>indispensable</i> · quality and time/cost overruns · the team-failure scenario · ad hoc development is a sure way to fail · "when to do what" · programming-in-the-small vs -in-the-large
Mall p.71	Why document a development process · the four named problems · code reviews and test-case timing as examples · the management-seriousness signal

Master Studio · Advanced Software Engineering (CS504) · University of Wasit